

Tangle Cloud

The Decentralized Compute Layer for AI

Product Vision & Infrastructure

Tangle Foundation

June 2026

Abstract

Tangle Cloud is infrastructure for AI systems that improve themselves. Two products ship today: a sandbox runtime that gives any agent a real, isolated computer (shell, files, browser, any coding harness), and an agentic workbench that turns natural language into running, deployed software. Every run is traced, and those traces are mined to make the next run better. The same open-source tooling runs two ways: on Tangle Cloud’s managed infrastructure today, and on a network of independent operators as it comes online, so you get the ease of a managed product with no lock-in to it. This document describes the products, the infrastructure beneath them, the operator economics that let independent providers compete to run them, and the roadmap from managed service to decentralized compute market.

This document complements the Tangle Protocol Specification (core mechanisms and security model) and the Blueprint SDK Developer Guide (implementation details). Together they describe the protocol, SDK, and product layer.

Contents

1	Vision: Decentralized AI Infrastructure	3
1.1	The Product-Protocol Connection	3
1.2	Token Supply and Distribution	4
1.3	Network Effects	4
1.3.1	Supply Flywheel	4
1.3.2	Application Flywheel	4
1.3.3	Security Flywheel	4
1.3.4	Flywheel Interactions	5
2	The Decentralized Sandbox Runtime	5
2.1	Isolation Technologies	5
2.2	Isolation Guarantees	5
2.3	Deployment Models	6
2.3.1	Decentralized Operators	6
2.3.2	Managed Cloud	6
2.4	Sandbox as Sidecar	6
2.5	Pricing Mechanisms	6

3	The Agentic Workbench	7
3.1	Natural Language Development Platform	7
3.2	Session Persistence	7
3.3	Technical Architecture	7
3.4	Collaborative Features	8
3.5	AI-Assisted Knowledge Work	8
3.6	The Parallel Agents Paradigm	8
3.7	The Evaluation Loop	9
4	Product Surface and Extensions	9
4.1	Product Interactions	9
4.2	Third-Party Extensions	9
4.3	Use Cases	9
5	Incentives and Pricing	10
5.1	Operator Economics	10
5.2	Customer Pricing	10
5.2.1	Pricing Examples	10
5.2.2	Pricing Models	11
5.3	TNT Token Utility	11
5.4	Governance Parameters	11
6	Roadmap	12
6.1	Current State	12
6.2	Near-Term Priorities	12
6.3	Longer-Term Development	13
6.4	Dependencies	13
6.5	Adaptability	13
7	Measuring Success	13
8	Risks and Limitations	14
8.1	Technical Risks	14
8.2	Economic Risks	14
8.3	Adoption Risks	14
8.4	Competition	15
9	Conclusion	15

1 Vision: Decentralized AI Infrastructure

AI systems are starting to do real economic work: writing and shipping code, running long workflows, and making decisions with money attached. That work needs somewhere to run: a real computer per agent, isolation you can trust, and a record of what happened so the next run is better.

Tangle Cloud provides that infrastructure. Two products ship today. A sandbox runtime gives each agent an isolated computer. An agentic workbench turns natural language into deployed software. Every run they produce is traced and mined for improvement. This is infrastructure for AI that gets better the more it runs.

The infrastructure is built to decentralize. Centralized clouds work, but they set prices unilaterally, define terms of service, choose which customers to serve, and capture most of the economics their products create. For infrastructure that may run significant economic activity, that concentration is a real risk. Tangle Cloud runs the same open-source tooling two ways: on managed infrastructure today, and on a network of independent operators as it comes online. Prices emerge from a market rather than a corporate decision. Value flows to the operators, developers, delegators, and keepers doing the work. And because the stack is open, you are never locked into any one provider, including us.

1.1 The Product-Protocol Connection

Every interaction with Tangle Cloud products generates economic activity that accrues to network participants:

Customers use products (sandbox runtime, workbench, or third-party applications), request services from operators, and pay in tokens. Payments split among developers, operators, and the protocol.

Operators provide compute by staking tokens and running infrastructure. They earn fees proportional to services provided, and can serve more services as they accumulate more stake.

Developers create blueprints defining service behavior and earn from fee splits and emission rewards proportional to adoption.

Delegators provide additional stake to operators and share in operator rewards proportional to their delegation.

Protocol captures a governance-controlled fee (currently 19.5%, with a 0.5% keeper rebate carved out so permissionless billing is EV-positive) funding development, security audits, and network operations.

Service payments split across five recipients in the service’s payment asset, as summarized in Table 1.

Table 1: Canonical service fee split (sums to 100%).

Recipient	Share
Operator	40%
Developer	20%
Delegators (stakers)	20%
Protocol	19.5%
Keeper rebate	0.5%

The 0.5% keeper rebate is carved from the protocol share so that permissionless subscription billing is EV-positive for whoever triggers it.

1.2 Token Supply and Distribution

TNT is hard-capped: `MAX_SUPPLY` = 100,000,000 TNT. Genesis mints the full cap across four buckets (Substrate $\approx 51.24\text{M}$, EVM $\approx 1.13\text{M}$, Treasury $\approx 32.59\text{M}$, Foundation $\approx 15.04\text{M}$), so `totalSupply` equals the cap at genesis. The protocol is hard-capped; the `InflationPool` cannot mint; emission is treasury-funded; only governance can mint and only up to the cap (no headroom at genesis). “Emission” is treasury-funded: $\approx 1\%$ of supply per year ($\approx 1,000,000$ TNT) is transferred from the Treasury bucket into the `InflationPool`; it dilutes no one and creates no new supply.

Migrated balances are claimed via an SP1 Groth16 zk-proof of Substrate key ownership plus a merkle proof. Vesting unlocks 10% at claim, then a 12-month cliff and 24-month linear release; treasury and foundation allocations vest over ≈ 3 years (foundation with 30% unlocked at start). See the *Tangle Protocol Specification* for the full supply and distribution schedule.

1.3 Network Effects

Tangle exhibits positive network effects that compound over time. Understanding these dynamics helps participants identify opportunities and predict growth.

1.3.1 Supply Flywheel

More operators $\rightarrow$ more compute $\rightarrow$ more products $\rightarrow$ more demand $\rightarrow$ more fees $\rightarrow$ more operators.

As operator count increases, geographic and capability diversity grows. Customers find operators meeting their specific requirements. More customer options attract more demand. More demand generates fees that attract operators with capital to stake.

The flywheel accelerates as barriers fall. Improved tooling reduces operational burden. Documentation and support lower entry costs. Reference implementations demonstrate viability.

1.3.2 Application Flywheel

More developers $\rightarrow$ more blueprints $\rightarrow$ more use cases $\rightarrow$ more customers $\rightarrow$ more fees $\rightarrow$ more developers.

Developers create blueprints when they see opportunity. Blueprint diversity attracts customers with varied needs. Customer payment generates fees and emission rewards for developers. Developer success attracts more developers.

Composability makes the application flywheel stronger. Blueprints can reference and extend each other. A successful oracle blueprint can feed DeFi blueprints. DeFi usage can pull in custody blueprints. Each useful component makes the next one easier to build.

1.3.3 Security Flywheel

More delegators $\rightarrow$ more stake $\rightarrow$ more security $\rightarrow$ higher-value use cases $\rightarrow$ more fees $\rightarrow$ more stake.

Security is the product delegators produce. More stake backing operators enables higher-value services. Required security is set per service via exposure commitments and the 50% per-service slash ceiling, not a fixed ratio. Higher-value services generate more fees. Fees attract more delegation.

Delegation yield compounds the effect. Delegators earning attractive yield retain and increase positions. Growing stake enables progressively higher-value use cases. Each tier of security unlocks new customer segments.

1.3.4 Flywheel Interactions

These flywheels interact in reinforcing cycles:

- More security attracts enterprise customers requiring strong guarantees
- Enterprise customers demand specialized blueprints for their industries
- Specialized blueprints require capable operators with domain expertise
- Capable operators attract delegations from participants seeking yield
- Delegations increase security, enabling more enterprise customers

The network grows as a coherent whole rather than isolated components. Early participants benefit from positions in nascent flywheels. Later participants benefit from mature infrastructure and proven economics.

2 The Decentralized Sandbox Runtime

The sandbox runtime is where autonomous work actually executes. It provides secure, accountable compute for AI agents and automated workflows.

The sandbox runtime implements decentralized AI agent execution. Operators run agents in isolated containers with cryptoeconomic accountability. The protocol coordinates without controlling, addressing the concentration concerns outlined in Section 1.

2.1 Isolation Technologies

Operators host sandbox environments using industry-standard isolation:

Docker provides container-based isolation with process, filesystem, and network separation. Containers are lightweight, well-understood, and widely deployed. Suitable for many workloads where standard isolation suffices.

gVisor adds an additional isolation layer. gVisor intercepts system calls, providing a user-space kernel that limits attack surface. Suitable for higher-security requirements where container escapes are a concern.

Firecracker and micro VMs provide hardware-level isolation. Micro VMs boot in milliseconds while providing VM-strength isolation. Suitable for workloads requiring the strongest guarantees, where even kernel-level exploits should not compromise the host.

Operators choose technologies based on their security requirements, performance needs, and the blueprints they serve. The protocol provides the economic framework; operators make technical decisions.

2.2 Isolation Guarantees

Regardless of technology, certain guarantees must hold:

- **Process isolation:** No access to host resources or other sessions
- **Filesystem isolation:** Private storage with quotas
- **Network isolation:** Restricted external access per policy
- **Resource limits:** Bounded CPU, memory, and other consumption

Operators who fail to maintain guarantees face slashing. Slashing is bounded and contestable: the per-service penalty is capped at 50% (a hardcoded 100% ceiling, governance-raisable per service for equivocation), a challenger posts a 0.02 ETH dispute bond (refunded on cancel, forfeited on execute), and disputes resolve within a 21-day window that fails open (auto-executes if unresolved). An operator with any pending slash is frozen from unstaking, preventing evasion.

2.3 Deployment Models

2.3.1 Decentralized Operators

Independent operators stake assets, run infrastructure, and compete for customers. They earn fees proportional to services provided. They choose their technology stack, geographic location, and pricing strategy.

Benefits:

- No single point of failure
- Competitive pricing from market dynamics
- Geographic and organizational diversity
- Economic accountability through stake

2.3.2 Managed Cloud

For customers preferring a managed experience, Tangle offers hosted infrastructure with the same protocol guarantees. The managed option provides:

- Simplified onboarding
- Consistent SLAs
- Integrated billing
- Technical support

Both models use the same protocol. Customers can mix operators, some decentralized, some managed, based on their requirements.

2.4 Sandbox as Sidecar

The sandbox can run as the primary execution environment or as a sidecar alongside other systems:

- DeFi protocols for AI-assisted monitoring
- Oracle networks for data processing
- Keeper networks for decision logic
- Any system needing secure, accountable AI execution

2.5 Pricing Mechanisms

Service pricing uses request-for-quote (RFQ):

1. Customer broadcasts quote request specifying requirements
2. Operators return EIP-712 signed quotes with pricing
3. Customer accepts selected quotes on-chain, where signatures are verified
4. Service activates at the committed prices

This fits an operator market where hardware, location, uptime, isolation, and model access differ by provider. Customers compare the full quote, not price alone.

For recurring workloads, subscription pricing provides predictable costs. Customers fund escrow; anyone can permissionlessly trigger TWAP-fair billing at intervals, earning the 0.5% keeper rebate.

For variable workloads, event-driven pricing charges per job. Customers pay when they submit work.

3 The Agentic Workbench

The workbench is where autonomous work is authored and refined. It provides the creative environment for designing, testing, and deploying AI-powered workflows.

3.1 Natural Language Development Platform

Today, the workbench is an AI-assisted development product for blockchain projects. Users describe intent in natural language; agents translate that intent into implementation.

The product handles development environment setup. Pre-configured containers include SDKs, tools, and dependencies:

- **Ethereum:** Foundry, Hardhat, OpenZeppelin
- **Solana:** Anchor, Solana CLI, program scaffolding
- **Other chains:** Configured on demand

Users focus on what they want to build, not environment setup.

3.2 Session Persistence

Sessions persist across interactions. Each maintains:

- The codebase
- Conversation history
- Agent's accumulated project context

Users return to ongoing projects, review agent work, provide feedback, and iterate. Development becomes human-AI collaboration where the agent remembers previous decisions.

3.3 Technical Architecture

The current workbench implementation uses a centralized client-server architecture:

1. User initiates coding session via the web application
2. An orchestrator service provisions and manages sandbox containers
3. WebSocket connections carry user inputs to the agent and stream outputs back
4. The orchestrator handles container lifecycle, auto-scaling, and resource allocation
5. A collaboration server enables real-time multiplayer features
6. Protocol handles payment splits (see Table 1)

The orchestrator manages container provisioning, health monitoring, and session routing. This centralized architecture provides reliable performance and simplified operations during the current phase of development.

Target architecture. The roadmap calls for progressive decentralization of the workbench infrastructure. In the target architecture, session provisioning will use the protocol's RFQ

marketplace for operator selection, operators will stake assets and compete to serve workbench sessions, and P2P communication (gossipsub) will replace the centralized WebSocket relay. This transition will bring the workbench in line with the cryptoeconomic accountability model described in the *Tangle Protocol Specification*. The centralized orchestrator will be replaced by protocol-level coordination, where blueprints define the workbench service, operators stake to provide compute, and slashing conditions enforce isolation guarantees.

3.4 Collaborative Features

The workbench supports multiplayer collaboration:

- Teams work on shared projects with synchronized state
- Multiple people observe agent progress
- Coordinated input on complex tasks

Unlike traditional IDE collaboration (editing same files), workbench collaboration means directing the same agent infrastructure: multiple humans guiding multiple agents toward shared goals.

Parallel development: One team member defines architecture while another refines implementation. A third focuses on testing. Agents work in parallel, each supervised by the relevant expert.

Collective oversight: AI agents make mistakes. Teams collectively review outputs, catch errors, and guide refinement. Shared oversight improves quality beyond what individuals achieve.

Knowledge transfer: Junior developers work alongside seniors, observing how experienced engineers direct agents. The workbench becomes a training environment.

3.5 AI-Assisted Knowledge Work

The workbench expands beyond code to general knowledge work:

Research agents gather, synthesize, and present information. Deploy an agent to analyze competitors, summarize reports, identify regulations, and present findings.

Analysis agents process data and generate insights. Upload a dataset; the agent performs statistics, identifies patterns, generates visualizations, and suggests interpretations.

Writing agents produce reports, documentation, and communications. Transform analysis into polished output: summaries, documentation, presentations.

The vision is a unified environment for all AI-assisted work. Users engage with agents across task types without switching tools.

3.6 The Parallel Agents Paradigm

Traditional AI interfaces present single-threaded conversation. Real projects involve exploration, comparison, and parallel investigation.

Spawning sub-agents: A primary agent working on a smart contract spawns sub-agents to research gas optimization, investigate similar implementations, and draft tests. Each runs independently, reporting results back.

Forking for exploration: Facing an architectural decision, fork the context and direct each down a different path. Both develop in parallel. Observe progress, compare results, pick the winner.

Spatial canvas: The workbench presents parallel activity visually, showing all active agents, their status, recent outputs, and relationships. Users manage parallel work without overwhelm.

3.7 The Evaluation Loop

Every execution generates traces: what agents did, inputs received, outputs produced, operation timing. These traces feed evaluation systems:

- Which prompt structures produce better results?
- Which model configurations work for which tasks?
- Which operators deliver lower latency?

This creates a flywheel: more usage → more traces → better system → more usage. Early users benefit from infrastructure; later users benefit from accumulated learning.

4 Product Surface and Extensions

The sandbox and workbench are first-party products. Third parties can build their own products on the same protocol.

4.1 Product Interactions

Users can design workflows in the workbench and deploy to sandbox runtime, or interact with sandbox directly through APIs. Enterprise systems, automated pipelines, and third-party applications access compute through standard interfaces.

The workbench itself consumes sandbox compute, making it simultaneously a product and a protocol customer.

4.2 Third-Party Extensions

The protocol supports:

- Specialized workbenches for domains (legal, medical, financial)
- Infrastructure tools for operators (monitoring, scaling, fleet management)
- Aggregator services helping customers find operators

Each third-party product creates and captures value while the protocol captures its 19.5% share regardless of which products generate activity.

4.3 Use Cases

Confidential AI for enterprises: A pharmaceutical company needs AI analysis of proprietary clinical data. They select operators with verified security credentials, require specific isolation, and rely on cryptoeconomic accountability (slashing-backed) for proper handling; hardware-attested confidentiality (TEE) is on the roadmap.

Accountable AI for high-stakes decisions: A trading firm needs confidence that analysis used the specified model and data. Operators are held accountable through stake and slashing; cryptographic verification of model and data integrity (ZK/TEE) is on the roadmap.

Collaborative AI for distributed teams: A research consortium spans institutions with different governance. Each runs operator infrastructure within its boundaries while participating in collaborative workflows.

Autonomous agents with accountability: Deploy AI agents that take real-world actions. Agents run in sandboxed environments with logged actions and economic guarantees.

Developer monetization at scale: Create a blueprint; earn from every service instance across all operators. Treasury-funded emission adds rewards proportional to adoption.

5 Incentives and Pricing

5.1 Operator Economics

Operators earn from multiple sources:

Service fees: Direct payment for services provided, distributed according to exposure commitment.

Emission rewards: A fixed 25% operator share of treasury-funded emission, distributed by stake weight and active-service participation. The exposure rail starts at 0 until live services and the fee distributor exist.

Tips and premiums: Customers may offer premiums for priority, specific hardware, or geographic proximity.

Operator profitability depends on:

- Hardware costs (compute, storage, network)
- Operational costs (maintenance, monitoring)
- Stake opportunity cost
- Market pricing dynamics

Treasury-funded emission is distributed across five buckets, summarized in Table 2. The stakers (exposure) bucket is the exposure rail and starts at 0% at genesis because live services and the fee distributor are not yet active (beacon-chain restaking and cross-L2 slashing are Phase 2; see Section 6).

Table 2: Emission allocation and distribution weights (sums to 100%).

Bucket	Weight
Staking	55%
Operators	25%
Customers	10%
Developers	10%
Stakers (exposure rail)	0%

5.2 Customer Pricing

Customers pay based on:

- Compute time and resources consumed
- Isolation technology required
- Operator quality and location preferences
- Service duration and commitment

The RFQ system enables price discovery. Customers can specify budgets; operators quote within those constraints.

5.2.1 Pricing Examples

AI Development Session (Workbench):

- Base compute: 2 vCPU, 4GB RAM container
- AI model inference: Pass-through from LLM provider
- Session persistence: Storage for code and context
- Rate: Competitive with centralized alternatives, with cryptoeconomic accountability

AI Agent Deployment (Sandbox Runtime):

- Resource allocation: Configurable CPU/memory/storage
- Isolation premium: +10-30% for gVisor/Firecracker vs Docker
- Geographic premium: Additional cost for specific region requirements
- Security requirement: High-value workloads require operators with larger stakes

5.2.2 Pricing Models

Four payment models accommodate different workload types:

Subscription: Predictable costs for recurring workloads. Customers fund escrow; anyone can permissionlessly trigger TWAP-fair billing at intervals, earning the 0.5% keeper rebate.

Event-driven: Per-job pricing for variable workloads. Customers pay when they submit work.

Pay-once: Upfront payment for one-time computations, distributed when operators approve.

RFQ quotes: Operator-signed EIP-712 quotes that the customer accepts on-chain at the committed price.

See the *Tangle Protocol Specification* for detailed payment flow mechanics.

5.3 TNT Token Utility

TNT has five protocol roles:

Staking is required for operator participation. Operators self-bond at least 1,000 TNT (the bond is always TNT; delegation may accept other assets, but the operator bond does not). The amount staked determines service capacity and lets operators accept higher-value workloads requiring greater economic security.

Delegation allows passive holders to earn yield by backing operators. The minimum delegation is 0.2 TNT. Delegators share in operator rewards (operator commission defaults to 10%, with a 7-day timelock on changes) proportional to their contribution. Longer lock commitments earn multipliers up to 1.6x (timestamp-based: 1.0/1.1/1.2/1.3/1.6x at 0/1/2/3/6 months). Unbonding takes ≈ 7 days for delegators and ≈ 14 days for operators. A liquid delegation vault (ERC-7540) is also available, with a 15% default operator commission.

Governance grants voting rights on protocol parameters. Token holders can propose and vote on fee structures, emission allocation and distribution weights, and protocol upgrades.

Payment is made in the service's chosen payment asset; staking, operator bonds, and governance are denominated in TNT. Fee discounts for paying in TNT are on the roadmap.

Fee distribution flows in the payment asset to recipients. Network activity pays the operators, developers, delegators, keepers, and protocol that support it.

5.4 Governance Parameters

At launch the governor uses an ERC-6372 timestamp clock (not block numbers). The proposal threshold is a flat 100,000 TNT ($\approx 0.09\%$ of supply), with a 1-day voting delay, a 7-day voting period, and a 4-day timelock applied uniformly to all executions (there is no separate upgrade tier). Quorum is set at 4% of total supply, deliberately low because most supply is vesting-locked at launch, leaving a thin delegatable float. A canceller guardian multisig and the 4-day timelock are the real capture defense; quorum ratchets up as vesting unlocks.

6 Roadmap

Development proceeds through phases targeting progressive capability and decentralization.

6.1 Current State

Core protocol contracts: Feature-complete and undergoing external security audit ahead of mainnet on Base. The full service lifecycle is implemented including: blueprint registration, operator staking, service creation via RFQ, job execution, payment distribution, and slashing with dispute windows. The contracts are a custom facet router behind a UUPS proxy (the fallback resolves a selector to one of the core mixin facets and delegatecalls into it over shared proxy storage); this is not an EIP-2535 Diamond (no `diamondCut`/loupe), and upgrades go through the UUPS path under governance. See *Tangle Protocol Specification* for mechanism details.

Blueprint SDK: Production-ready with the current feature set:

- Event triggers (blockchain events, cron schedules, custom conditions)
- Job handlers with extractor pattern (`TangleArg`, `TangleResult`)
- Result consumers for on-chain submission
- P2P networking (gossipsub, request-response, peer discovery)
- QoS-based slashing hooks (heartbeat, latency, availability)
- Background keepers for lifecycle automation

See *Blueprint SDK Developer Guide* for implementation details.

Workbench: AI-assisted development sessions are live. Sessions, environments, and basic collaboration are functional.

Operator network: Initial operators running sandbox infrastructure across multiple geographic regions.

6.2 Near-Term Priorities

Multiplayer collaboration: Real-time shared sessions enabling multiple users to direct agents simultaneously. Technical requirements: WebSocket state synchronization, conflict resolution for concurrent edits, role-based access control for team permissions.

Knowledge work expansion: Research, analysis, and writing tasks beyond code. Requires: new agent configurations with specialized prompts, expanded tool integrations (web search APIs, document processing libraries, data analysis frameworks), UI components for non-code outputs.

Operator network growth: Target 50+ operators across 10+ geographic regions within 6 months. Initiatives: operator onboarding documentation, one-click deployment scripts, monitoring dashboards, community support channels.

Additional isolation technologies:

- gVisor integration: User-space kernel for defense-in-depth against container escapes
- Firecracker micro-VMs: Hardware-level isolation with millisecond boot times
- Blueprint configuration: Operators declare supported isolation; customers specify requirements

Standard verification libraries: Audited, reusable implementations:

- Oracle verification (median aggregation, TWAP, threshold signatures)
- Compute verification (deterministic replay, model fingerprinting)
- Availability monitoring (heartbeat keepers, latency tracking)

6.3 Longer-Term Development

Full governance activation: Progressive decentralization transferring control to token holders. Phases:

1. Parameter governance (fee rates, emission allocation and distribution weights)
2. Upgrade governance (contract upgrades via timelock)
3. Full governance (guardian multisig sunset)

See *Tangle Protocol Specification*, Governance section for mechanism details.

Cross-chain deployment: Tangle Cloud launches on Base; cross-L2 slashing (OP-Stack native messenger) is Phase 2.

- Additional L2s: Arbitrum, Optimism for lower transaction costs
- Bridge integration: Canonical bridges for TNT token portability
- Service portability: Same blueprint deployable across chains

Ecosystem grants: Fund development in key areas:

- Developer tooling (IDE plugins, debugging tools, testing frameworks)
- Verification research (ZK proofs for compute, TEE attestation)
- Operator infrastructure (monitoring, scaling, fleet management)

Advanced verification research:

- ZK proofs for compute verification without revealing inputs
- TEE integration (SGX, TDX) for hardware-attested execution
- Formal verification of protocol invariants

Protocol upgrades: Enhanced mechanisms based on operational learnings:

- Dynamic pricing algorithms responding to utilization
- Reputation systems for operator quality signaling
- Cross-service exposure limits for risk management

6.4 Dependencies

- Multiplayer requires stable single-user experience
- Knowledge work expansion requires proven agent reliability
- Governance activation requires sufficient token distribution
- Cross-chain requires mainnet stability

Near-term items are largely independent; longer-term items build on earlier foundations.

6.5 Adaptability

The roadmap will change as the operator market and customer demand become clearer. Governance adjusts priorities. Success is measured by operator count, service volume, developer adoption, and stake growth.

7 Measuring Success

Network health metrics:

Operator diversity measures geographic and organizational distribution. A healthy network has no single entity dominating compute provision.

Blueprint diversity tracks use case coverage. Blueprints should serve varied needs across industries and applications.

Customer growth monitors new customer acquisition and retention rates, indicating whether products deliver compelling value.

Fee volume measures total service-payment volume, of which the protocol takes a 19.5% slice, indicating real economic activity.

Stake growth tracks total value locked, reflecting increasing security capacity for higher-value workloads.

These metrics guide parameter adjustments: emission allocation, distribution weights, fee structures, and minimum stakes. The target is usage-backed growth, not emission-funded activity for its own sake.

8 Risks and Limitations

Honest assessment of challenges facing Tangle Cloud:

8.1 Technical Risks

Isolation breaches: Container and VM isolation technologies have known attack vectors. While defense-in-depth (Docker + gVisor + Firecracker) reduces risk, no isolation is perfect. Slashing provides economic deterrence but cannot undo data exposure.

Verification limitations: Some computations are inherently difficult to verify (AI model inference, complex simulations). Verification mechanisms may have false negatives (undetected cheating) or false positives (incorrect slashing). Blueprint designers must understand detection probability for their specific use case.

8.2 Economic Risks

Token volatility: Slashing effectiveness depends on stake value. Rapid token price declines can reduce security margins below safe thresholds faster than services can respond. Bounded slashing (the 50% per-service cap), the dispute window, and the pending-slash unstake freeze constrain the damage but cannot eliminate this risk.

Cold start problem: Two-sided marketplaces face bootstrapping challenges. Customers need operators; operators need customers. Early network growth is funded by treasury-funded emission, a finite runway ($\approx 1\%$ of supply per year) that must transition to sustainable fee-based economics before the treasury bucket is exhausted.

Thin delegatable float and quorum: At launch most supply is vesting-locked, so the governance quorum is set at only 4% of total supply. A canceller guardian multisig and the 4-day timelock are the primary capture defense until vesting unlocks and quorum ratchets up.

8.3 Adoption Risks

Operator supply: Decentralized infrastructure requires sufficient operators across geographies and capabilities. Operator economics must be attractive enough to compete with traditional employment or alternative yield opportunities.

Developer tooling maturity: The SDK and development experience must match or exceed centralized alternatives. Gaps in documentation, debugging tools, or deployment automation slow adoption.

8.4 Competition

Centralized cloud providers offer mature tooling, established trust, and economies of scale. Decentralized alternatives (Akash, Render) compete for similar users. Tangle’s differentiation through cryptoeconomic accountability must prove compelling enough to overcome switching costs.

9 Conclusion

Tangle Cloud is operational today, but unfinished. The sandbox runtime provides secure execution on distributed operator infrastructure. The workbench provides collaborative authoring for AI-assisted work. Together they show a path beyond centralized cloud dependence.

The products connect to the protocol’s economic layer. Every session, job, and agent interaction generates value that flows to operators, developers, delegators, and the protocol. Compute becomes a shared market rather than a single company’s margin line.

Centralized providers have already proven that AI infrastructure works. The open question is who controls it, who benefits, and whether credible alternatives exist. Tangle Cloud is that alternative: infrastructure that is resilient, competitive, and owned by its participants.

The work now is to make the operator market real at production scale.